Lösungen und Hinweise für die Lehrkraft

Arbeitsblatt „Wie viele Stammbrüche braucht man?“

Ablauf

Phase	Zeit	Inhalt
Einstieg	5 min	Ägypten, Papyrus Rhind; Bild des Papyrus zeigen
Teil A	8 min	Stammbrüche von Hand
Teil B	12 min	Greedy-Rezept (Fibonacci), Überraschung 5/121
Teil C	10 min	Rekordtabelle, log-Taste als „Stellenzahl“
Teil D	10 min	Erdős, Zeitleiste, 10 000 Stellen: **der Wow-Moment**
(45 min)		
Teil E	30–45 min	Kann man der KI trauen? Gruppendiskussion

Für 45 Minuten Teil E weglassen oder nur E1 kurz im Plenum besprechen.

Teil A

A1 1/3 + 1/15 = 5/15 + 1/15 = 6/15 = 2/5. 1/4 + 1/28 = 7/28 + 1/28 = 8/28 = 2/7.

A2 3/4 = 1/2 + 1/4; 5/6 = 1/2 + 1/3; 7/12 = 1/3 + 1/4 (auch 1/2 + 1/12).

Teil B

B1 3/7: Kehrwert 2,33 → 1/3; Rest 2/21. Kehrwert 10,5 → 1/11; Rest 1/231. **3/7 = 1/3 + 1/11 + 1/231.**

B2 10/11: 1/2, Rest 9/22; 1/3, Rest 5/66; 1/14, Rest 4/924 = 1/231. **10/11 = 1/2 + 1/3 + 1/14 + 1/231**, also **4** Stammbrüche. Kürzer geht es nicht (siehe Tabelle in Teil C); die kürzeste Zerlegung ist nicht eindeutig, z. B. auch 10/11 = 1/2 + 1/3 + 1/15 + 1/110.

B3 5 : 121 = 0,04132231…, und 1/25 + 1/759 + 1/208725 = 0,04132231… (stimmt exakt). Erkenntnis: Das Rezept funktioniert immer, liefert aber **nicht immer die kürzeste** Zerlegung, und die Nenner können **explodieren**. Weitere Beispiele: 4/17 liefert mit dem Rezept 4 Stammbrüche, kürzer ist 1/5 + 1/34 + 1/170; 77/79 liefert 7, kürzer geht es mit 6.

Teil C

C1 Die Zähler sind fast immer **Nenner minus 1**: Brüche **dicht bei 1** sind besonders schwer. (Grund: Man muss mit 1/2, 1/3, … anfangen, und der Rest bleibt lange groß.)

C2 Stellenzahlen 1, 1, 1, 2, 2, 2, 3, 5: Die Rekordnenner wachsen **immer schneller**, die Stellenzahl selbst wächst immer schneller (2 → 3 → 5).

C3

Stammbrüche	**4**	**5**	**6**	**7**	**8**
b	11	17	79	733	27 539
log(b)	1,04	1,23	1,90	2,87	4,44
log(log(b))	0,02	0,09	0,28	0,46	0,65

Ab 5 Stammbrüchen wächst log(log(b)) pro Stammbruch um etwa **0,19**. Anders gesagt: Pro „+1 bei der Stellenzahl der Stellenzahl“ kommen etwa 5 Stammbrüche dazu.

C4 Von 0,65 bis 2 sind es 1,35 ≈ 7 Schritte zu 0,19 → etwa **8 + 7 = 15** Stammbrüche für einen 100-stelligen Nenner. Betonen: **Schätzung**, nicht bewiesen. Genau hier setzt der Beweis an: Er garantiert, dass das Wachstum nie schneller als „ein festes Vielfaches von log(log(b))“ wird.

Herkunft der Tabelle: Die Rekordnenner 2, 3, 5, 11, 17, 79, 733, 27539 sind die OEIS-Folge A097048. Die Werte bis 733 wurden mit `optimum.py` aus diesem Repository unabhängig nachgerechnet (vollständige Suche).

Teil D

D1

Nenner	**S**	**S / log(S)**	**√S**	**log(S)**
100 Stellen	100	50	10	2
Googol	101	≈ 50,4	≈ 10,05	≈ 2,00
Googolplex	10^{100}	10^{98}	10^{50}	**100**

Beobachtung: Für riesige Nenner ist die neue Grenze winzig im Vergleich zu den alten. Selbst für einen Googolplex (eine Zahl, die man nicht einmal aufschreiben könnte, weil es nicht genug Atome im Universum für ihre Ziffern gibt) wächst die neue Grenze nur auf 100 (mal einen festen Faktor). *Hinweis:* Alle drei Grenzen gelten nur „bis auf einen festen Faktor“; welche Basis der Logarithmus hat, ändert nur diesen Faktor.

D2 Erdős’ Verfahren wächst gleichmäßig steil (bei 50 Stellen 78 Stammbrüche), das neue Verfahren bleibt fast flach (bei 50 Stellen 28). Bei 1000 Stellen braucht Erdős’ Verfahren 894 Stammbrüche (gemessen mit `erdos1950.py`); das neue Verfahren ist dort nicht mehr rechenbar, hochgerechnet etwa 45. Das neue Verfahren gewinnt also deutlich.

D3 Mit der Methode aus C4: 8 + (4 − 0,65) : 0,19 ≈ 8 + 18 ≈ **26** Stammbrüche (Schätzung). Zum Vergleich (Infokasten): Erdős’ Verfahren von 1950 liefert **6488** (gerechnet mit `erdos1950.py` für einen zufälligen 10 000-stelligen Nenner, a = b−1). Die bewiesene Untergrenze **14** stammt aus Erdős’ Argument: Braucht (b−1)/b nur k Stammbrüche, dann hat b höchstens so viele Stellen wie $(k+1)^{(2k)}$. Für k = 13 sind das 9390 Stellen, also reichen 13 Stammbrüche bei 10 000 Stellen nicht. *Für starke Schüler (optional):* Stellenzahl von $(k+1)^{(2k)}$ ≈ 2^k · log(k+1) mit dem Taschenrechner für k = 12, 13, 14 berechnen (4563, 9390, 19270). Unser Programm mit dem neuen Verfahren lässt sich bei 10 000 Stellen nicht mehr ausführen;

hochgerechnet aus den Messungen bis 55 Stellen ergäben sich etwa 60 Stammbrüche. **Wichtig für die Klasse:** „20 reichen“ oder „30 reichen“ kann derzeit niemand behaupten, weil der Beweis die feste Zahl nicht angibt. Das ist der Aufhänger für E3.

Teil E

E1 Es gibt **unendlich viele** Nenner. Ein Muster bei kleinen Zahlen kann später abbrechen. Nur ein Beweis deckt **alle** Nenner ab.

E2 Selbst prüfen: jeden Schritt nachvollziehen, Beispiele nachrechnen, Fachleute fragen. Bei zehn Seiten schwerer Mathematik ist das sehr aufwendig, bei über 700 Arbeiten kaum möglich. Das Prüfprogramm garantiert, dass **jeder Schritt logisch korrekt** ist. Es kann **nicht** prüfen, ob die Aufgabe richtig in die Computersprache übersetzt wurde (das müssen Menschen kontrollieren), und es erklärt nicht, **warum** etwas stimmt, also das Verstehen. *Hintergrund:* Das Prüfprogramm heißt Lean (ein sogenannter Beweisassistent). Für dieses Ergebnis ist die Aufgabenstellung in Lean nur etwa 30 Zeilen lang; das OpenAI-Repository enthält die vollständige, geprüfte Lean-Fassung des Beweises.

E3 Ja, ein Fortschritt: Vorher war nur bewiesen, dass die Anzahl höchstens wie „Stellenzahl geteilt durch Stellenzahl der Stellenzahl“ (Erdős) bzw. wie die Wurzel der Stellenzahl (Vose) wächst, also bei 10 000 Stellen etwa 2500 bzw. 100 (jeweils mal einem festen Faktor). Jetzt ist klar, dass das Wachstum so langsam ist wie überhaupt möglich (die Untergrenze hat dieselbe Form). Das ist eine Aussage über **alle** Nenner auf einmal.

Zusatzinfo für die Diskussion: Verfolgt man die Zahlen im Beweis, garantiert er grob „höchstens etwa **10 Milliarden** mal log(log(b))“, und das auch erst für Nenner, deren Stellenzahl selbst unvorstellbar viele Stellen hat (grob b ab exp(exp(2^(2 450 000))); eigene Abschätzung, nicht im Paper angegeben). Bewiesen ist auch die Untergrenze: Für (b−1)/b braucht man mindestens etwa 0,59 · log(log(b)) Stammbrüche (natürlicher Logarithmus). Praktisch reichen nach unseren Messungen etwa 6 · log(log(b)), die exakten Rekorde liegen bei etwa 3,4 · log(log(b)). Kontrast für die Klasse bei 10 000 Stellen (natürlicher Logarithmus: log(log(b)) ≈ 10): „vom Beweis garantiert: höchstens rund 100 Milliarden – vermutlich: etwa 26 – sicher nötig: 14“. Der Beweis liefert die richtige *Wachstumsart*, nicht die richtige *Zahl*.

Offen bleibt die konkrete Zahl: Mathematik kennt viele solche Ergebnisse („es gibt eine Konstante …“), und oft wird die Konstante erst in späteren Arbeiten verbessert oder berechnet.

E4 Offene Diskussion. Mögliche Impulse:

- Die KI baut auf Jahrhunderten menschlicher Mathematik auf (Erdős, Vose und viele andere). Im Paper werden diese Vorarbeiten zitiert. Als Autor ist „OpenAI“ angegeben.
- Hunderte Arbeiten auf einmal: Fachleute können nicht alles prüfen; Prüfprogramme helfen, gibt es aber bisher nur für einen Teil.
- Selbst Mathematik zu können bleibt nötig, um Ergebnisse zu verstehen, zu beurteilen und die richtigen Fragen zu stellen.

Hintergrund (für die Lehrkraft)

- **Satz (OpenAI, 25.09.2026, „Short Egyptian fractions“):** Es gibt Konstanten c_1, $c_2 > 0$, so dass für alle großen b gilt: $c_1 \cdot \log\log b \le N(b) \le c_2 \cdot \log\log b$, wobei N(b) die größte Mindestanzahl verschiedener Stammbrüche für einen Bruch a/b ist.

- **Kernidee:** Man wählt eine Hilfszahl M mit sehr vielen Teilern (Produkte von Primzahlen). Mit jedem Stammbruch kann man den Zähler auf einen Rest modulo einem passenden Teiler verkleinern. Weil es so viele Teiler gibt, ist ein sehr kleiner Rest dabei, und nach wenigen Schritten ist man fertig.
- **Quellen:** OpenAI-Repository github.com/openai/math, Preprint `preprints/Short-Egyptian-fractions-September-25-2026`, Lean-Formalisierung `lean/docs/025.md`; Erdős-Problem Nr. 304 auf erdosproblems.com; OEIS A097048.
- **Zum Vorführen** (optional, Python): im Hauptverzeichnis dieses Repositorys
  - `python3 optimum.py 77 79`: kürzeste Zerlegung,
  - `python3 stammbruch_loglog.py 77 79 --alle`: Zerlegung nach dem neuen Beweis,
  - `python3 erdos1950.py --stellen 10 50 1000`: Erdős' Verfahren.